

LLM Council v4.0 (Enterprise Edition)

Multi-agent AI debate system. 4 LLMs argue. 1 judge decides. Voice, image & webcam supported.

Project Documentation

Comprehensive project documentation has been moved to the `docs/` directory to keep the root repository clean:

- [Product Requirements Document \(PRD\) \(./docs/PRD.md\)](#)
- [Technical Requirements Document \(TRD\) \(./docs/TRD.md\)](#)
- [Project Details \(./docs/Project_Details.md\)](#)
- [Project Report \(./docs/Project_Report.md\)](#)

Quick Start

1. Get your free OpenRouter API key

→ openrouter.ai/keys (<https://openrouter.ai/keys>) (free, no credit card needed)

2. Setup `.env`

```
copy .env.example ml_engine/.env
# Edit ml_engine/.env - paste your OPENROUTER_API_KEY
# (Optional) Add JWT_SECRET_KEY for persistent sessions
```

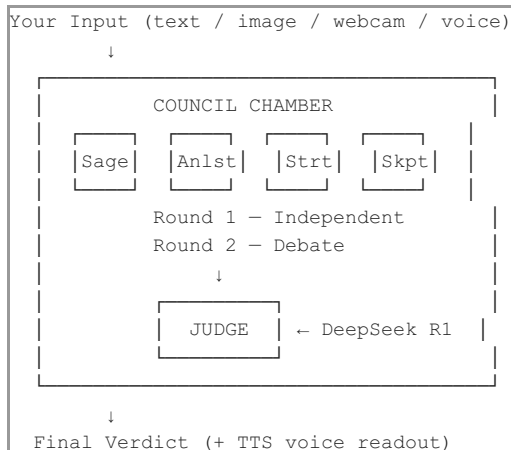
3. Run

All startup scripts have been moved to the `scripts/` folder.

```
cd scripts
start.bat           # Windows - one-click launch for both Frontend & Backend
```

Open <http://localhost:3000> in Chrome (for full voice support).

How It Works



Custom Agents & Council Members

In addition to the default Council (Sage, Analyst, Strategist, Skeptic), you can now create **Custom Agents** via the UI.

Custom agents are persisted in the SQLite database and allow you to define specific System Prompts and roles to tailor the debate to your exact needs.

Architecture

- **Backend:** FastAPI + LangGraph + WebSockets.
- **Frontend:** Next.js (React) + Tailwind CSS.
- **Database:** SQLite (User Auth & Config).

- **Execution:** Hybrid Adapter supports both **Cloud** (OpenRouter) and **Local** (.gguf) models.
- **Memory:** ChromaDB for Vector Search and Precedent saving.

Security & Enterprise Features

This project has been hardened against common web exploits:

- **Rate Limiting:** Integrated `slowapi` to protect authentication endpoints from brute-force dictionary attacks (5 requests/min per IP).
- **Payload Limits:** Hard limits on WebSocket data streams (5MB max) to prevent memory exhaustion (DoS) via malicious file uploads.
- **Strict Validation:** Pydantic `Field` bounds enforce data sanitization and prevent database bloating on all API endpoints.
- **Secure Secrets:** Securely randomizes the JWT Secret Key via Python's `secrets` module if missing from `.env`, with shortened token lifespans (60 minutes) and strict regex password complexity requirements.

Repository Structure

- `frontend/` — Next.js React application
- `ml_engine/` — FastAPI backend, LangGraph logic, and SQLite database
 - `ml_engine/tests/` — Automated test suite
- `docs/` — Core architecture and design documentation
- `scripts/` — Batch scripts for environment setup and application launching

Input Modes

- **Text** — Type any question or topic
- **Image** — Upload a photo; vision agents analyze it
- **Webcam** — Capture a live frame from your camera
- **Voice** — Speak your question (uses Chrome's Web Speech API)

Output

- Real-time streaming debate via WebSockets.
- Judge's verdict with **HIGH / MEDIUM / LOW** confidence.
- **Text-to-speech** readout of the verdict (JARVIS-style).
- Option to **branch** timelines mid-debate.

Troubleshooting

- Use **Chrome desktop** for best Web Speech and camera behavior.
- Run on `localhost` or `127.0.0.1` (secure context required for media APIs).
- If the backend returns connection errors, check that the scripts successfully booted both the `8001` FastAPI port and the `3000` Next.js port.

Project Details

Project Name: LLM Council (v4.0 Enterprise Edition)

Overview

LLM Council is a highly experimental, sophisticated multi-agent system designed to bring the power of diverse LLM perspectives to the end-user. Instead of relying on a single AI model to answer a prompt, LLM Council orchestrates a debate between four distinct AI personalities, followed by a final verdict from a "Judge" agent. The system now supports Custom Agents defined by the user to tailor debates to highly specific use-cases.

Motivation

As LLMs become more capable, their biases, limitations, and "hallucinations" remain a challenge. By having multiple state-of-the-art models (Llama, Qwen, Gemini, Mistral, DeepSeek) debate each other, the system forces rigorous fact-checking, strategic planning, and philosophical grounding. The result is a much more robust, well-rounded answer than any single model could provide.

With version 4.0, the motivation expanded to include creating a secure, enterprise-grade architecture capable of defending against malicious inputs, credential stuffing, and resource exhaustion.

Team

- **Creator/Lead Developer:** AMMAR (BEAST-AMMAR)
- **AI Assistant:** Google DeepMind Agent (Antigravity)

Timeline & Milestones

- **v1.0:** Initial command-line prototype of multi-agent debate.
- **v2.0:** Introduction of the JARVIS-style frontend UI with basic WebSocket streaming.
- **v3.0:**
 - Complete migration to FastAPI backend and Next.js frontend.
 - Implementation of Hybrid Model Adapter (Cloud OpenRouter + Local GGUF execution).
 - Advanced multimodal capabilities (Voice API, WebCam capture, Image uploading).
- **v4.0 (Enterprise Edition - Current):**
 - Total security overhaul: implemented `slowapi` rate limiting, WebSocket payload bounds, and strict Pydantic input validation.
 - Hardened JWT lifecycle with secure fallback runtime secrets.
 - Complete folder structure reorganization (moved tests, scripts, and docs into dedicated directories).
 - UI-driven Custom Agent creation with database persistence.
 - Robust search fallback layer (DuckDuckGo -> Tavily API) for agent fact-checking.

Technological Highlights

- **LangGraph Integration:** State-of-the-art graph-based AI orchestration allows complex routing of context between agents without messy imperative code.
- **Real-Time Streaming:** The integration of Python `asyncio` and FastAPI WebSockets ensures that the user doesn't wait minutes for the debate to finish; they read the debate in real-time token-by-token.
- **Cost Efficiency & Privacy:** Utilizing OpenRouter's free tier models ensures that the application is accessible and completely free to run in the cloud, while local `.gguf` fallback ensures absolute privacy and offline capability.
- **Enterprise Security:** Built to withstand brute-force login attempts, massive file drop payloads, and SQL injection via strict ORM usage.

Product Requirements Document (PRD)

Project Name: LLM Council (v4.0 Enterprise Edition)

1. Introduction

LLM Council is a multi-agent AI debate system where specialized LLM agents discuss and debate a user-provided prompt or topic, while a "Judge" agent oversees the debate, synthesizes arguments, and delivers a final verdict. The product allows users to gain diverse perspectives on complex problems before arriving at a well-reasoned conclusion. Version 4.0 elevates this to an enterprise standard with robust security, rate-limiting, and custom agent capabilities.

2. Objectives

- Provide users with highly nuanced, well-debated answers to complex queries.
- Support multimodal inputs (Text, Voice, Image, and live Webcam Video) to make the council accessible and versatile.
- Deliver an interactive "JARVIS-like" user interface where users can watch the AI agents deliberate in real-time.
- Offer local and cloud model execution capabilities.
- **Provide enterprise-grade security against DoS attacks, brute-force attempts, and memory exhaustion.**
- **Allow dynamic creation and persistence of Custom Agents via the UI.**

3. Target Audience

- Researchers and professionals looking for deep analysis of complex topics.
- Developers and tech enthusiasts interested in multi-agent LangGraph architectures.
- General users seeking an advanced alternative to single-model chatbots.
- **Enterprise teams needing secure, rate-limited AI deployment.**

4. Key Features

4.1. The Council (Multi-Agent System)

- **Sage (Philosopher):** Focuses on ethical, philosophical, and long-term implications.
- **Analyst (Logician):** Focuses on data, logic, factual correctness, and structural breakdown.
- **Strategist (Visionary):** Focuses on actionable plans, future trends, and out-of-the-box thinking.
- **Skeptic (Challenger):** Acts as the devil's advocate, finding flaws in the other agents' reasoning.

- **Judge:** Synthesizes the debate, issues a final verdict, provides a confidence score, and determines if a rerun is necessary.
- **Custom Agents:** Users can define their own agents (Name, Role, System Prompt) that persist in the database and can be swapped into the debate.

4.2. Multimodal Inputs

- **Text:** Standard keyboard input.
- **Voice:** Speech-to-text integration via browser Web Speech API.
- **Image Upload:** Users can drop files into the context.
- **Live Video Stream:** Captures webcam frames to pass vision context to the council.

4.3. Real-Time Streaming & Interaction

- The UI streams the agents' internal thoughts ("thinking") and final tokens in real-time via WebSockets.
- **Branching Timelines:** Users can branch off the debate with a hypothetical scenario ("what if X was true?") to see how the council reacts.

4.4. Audio Output

- **Text-to-Speech (TTS):** The final verdict from the Judge is read aloud.

4.5. Hybrid Model Support

- Users can toggle between **Cloud Mode** (OpenRouter APIs for Llama 3, Qwen, Gemini, DeepSeek, Mistral) and **Local Mode** (using locally downloaded weights for privacy).

4.6. Enterprise Security

- Strict `slowapi` rate limiting on auth routes.
- Strict payload limits (5MB) on WebSockets to prevent memory exhaustion.
- Pydantic input validation to prevent buffer overflows and DB bloat.

5. Non-Functional Requirements

- **Performance:** Real-time WebSocket streaming with minimal latency.
- **Reliability:** Fallback systems for API timeouts (e.g., DuckDuckGo -> Tavily search fallback).
- **Scalability:** Stateless WebSocket design allowing for multiple concurrent debate sessions.
- **Security:** Local SQLite database for session and user authentication. Token-based auth for secure access with strict expiration policies.

6. Future Enhancements

- Integration of custom user-defined agents (Completed).
- Saving debate transcripts and exporting to PDF (Completed).
- Persistent user history across devices (Completed).
- Migration from SQLite to PostgreSQL for distributed scaling.

Technical Requirements Document (TRD)

Project Name: LLM Council (v4.0 Enterprise Edition)

1. System Architecture

The application is built using a decoupled, highly secure architecture:

- **Backend:** FastAPI (Python) handles API routes, WebSocket streaming, authentication, rate-limiting, and the AI orchestration.
- **Frontend:** Next.js (React) handles the user interface, media capture (camera, mic), and WebSocket communication.
- **AI Orchestration:** LangGraph coordinates the multi-agent debate flow, passing state between agents.
- **Database:** SQLite (via SQLAlchemy) stores user credentials, custom agents, and configuration state.
- **Vector Store:** ChromaDB is used for long-term memory retrieval and storing past debate precedents.

2. Tech Stack

2.1 Backend

- **Framework:** FastAPI
- **Server:** Uvicorn

- **AI Framework:** LangChain & LangGraph
- **ORM:** SQLAlchemy
- **Database:** SQLite (`llm_council.db`)
- **Authentication:** JWT tokens (OAuth2 Password Bearer) with strict 60-minute expiration.
- **Security:** `slowapi` for API rate-limiting. Pydantic models for strict input validation.
- **WebSockets:** Native FastAPI WebSockets for real-time streaming of tokens and media context, with 5MB payload limits.

2.2 Frontend

- **Framework:** Next.js (React)
- **Styling:** Tailwind CSS / Vanilla CSS
- **Media APIs:** HTML5 Canvas, `getUserMedia` (Video), Web Speech API (Audio/Speech recognition).
- **Communication:** HTTP REST (Auth, Health, Agents), WebSockets (Debate streaming).

3. Core Components

3.1 LangGraph Debate Engine (`debate_graph.py`)

- Manages the **state of the debate** (`messages`, `round_count`, `max_rounds`, `current_topic`, `session_id`, `personality`).
- Agents are nodes in the graph. The edges dictate the flow: User Prompt -> Independent Agent Analysis -> Debate Round -> Judge Verdict.
- Includes a robust **search fallback mechanism** (DuckDuckGo -> Tavily).
- Integrated `stream_callback` to send intermediate tokens via WebSocket to the frontend.

3.2 Hybrid Model Adapter (`adapter.py`)

- **Cloud Mode:** Uses the `openai` SDK pointing to OpenRouter's API endpoints to route queries to free tier models (Llama 3, Qwen, Gemini, DeepSeek, Mistral).
- **Local Mode:** Loads models directly from the local filesystem (`/models/*.gguf`) for offline, private execution.
- Handles model fallback logic gracefully.

3.3 WebSocket Manager (`server.py`)

- Establishes persistent connections per client with memory-leak prevention.
- Routes actions based on JSON payloads:
 - `convene`: Starts a standard debate.
 - `branch`: Injects alternative context into the debate.
 - `audio_stream / video_stream / file_drop`: Stores multimodal context in a global dictionary keyed by `session_id`.

3.4 Security & Authentication Module (`auth.py`, `limiter.py`, `database.py`)

- Standard JWT implementation with secure runtime secret generation via Python's `secrets` module.
- Strict Pydantic validators enforce password complexity and string length bounds.
- `slowapi` decorators protect endpoints from brute-force attempts.
- SQLAlchemy models for Users and Custom Agents.

4. API Endpoints

- **REST:**
 - GET `/api/health`: Returns server status and current adapter mode.
 - POST `/api/mode`: Sets the hybrid adapter mode (`local` or `cloud`).
 - POST `/api/auth/token`: Generates JWT tokens for authentication (Rate-Limited).
 - POST `/api/auth/register`: Registers a new user (Rate-Limited).
 - GET `/api/agents` & POST `/api/agents`: Manage custom agents.
- **WebSocket:**
 - `ws://<host>:<port>/ws/debate/{room_id}`: Main bidirectional stream.

5. Deployment & Execution

- **Environment:** Configured via `ml_engine/.env`.
- **Startup Script:** Moved to `scripts/start.bat` for Windows users to easily launch the Uvicorn server and Next.js frontend concurrently.
- **Port:** Defaults to 8001 for the backend, 3000 for the frontend.

Project Report

Project Name: LLM Council (v4.0 Enterprise Edition)

1. Executive Summary

LLM Council successfully bridges the gap between single-model querying and comprehensive multi-model reasoning. The release of version 4.0 has significantly enhanced the system architecture, graduating it from a sophisticated prototype to an enterprise-grade, highly secure FastAPI/Next.js application capable of real-time WebSocket communication, custom agent injection, and robust defense against web vulnerabilities.

2. Objectives Achieved

- **Multi-Agent Orchestration:** Successfully implemented a 5-agent debate system using LangGraph. Agents (Sage, Analyst, Strategist, Skeptic, and Judge) effectively pass context, maintain individual personalities, and synthesize final verdicts.
- **Enterprise Security Overhaul:** Secured the application with `slowapi` rate limiting, WebSocket memory exhaust prevention (5MB payload limits), and strict JWT/Pydantic validation schemas.
- **Custom Agent Architecture:** Implemented a fully dynamic Custom Agent pipeline, allowing users to configure bespoke agents with tailored system prompts directly from the UI, persisting them securely in SQLite.
- **Multimodal Context:** Integrated HTML5 canvas and Web Speech API into the Next.js frontend, allowing the backend to receive image attachments, live video frames, and transcribed audio as debate context.
- **Hybrid Execution:** Built the `hybrid_adapter.py` layer, which dynamically routes requests either to OpenRouter's cloud APIs or to local `.gguf` weights, providing users with the choice between high-speed cloud execution and absolute local privacy.
- **Real-Time Streaming:** Resolved previous latency issues by utilizing FastAPI WebSockets and LangGraph's streaming capabilities, ensuring the user interface updates instantly as tokens are generated.

3. Technical Challenges & Solutions

3.1 Synchronous vs Asynchronous Graph Execution

Challenge: LangChain and LangGraph default to synchronous execution which blocks the FastAPI event loop, causing the WebSocket to drop connection or fail to receive intermediate tokens.

Solution: Migrated the graph invocation to use `ainvoke` and wrapped the execution in `asyncio.create_task()`. Implemented custom async stream callbacks to push tokens to the active WebSocket session seamlessly.

3.2 Handling Large Multimodal Payloads via WebSockets

Challenge: Sending base64 video frames or massive file drops repeatedly via WebSockets caused buffer bloat and server lag, leading to potential DoS vulnerabilities.

Solution: Enforced a strict 5MB payload cutoff inside the WebSocket receiver loop. Optimized the frontend to only capture and send keyframes when the user explicitly triggers a "capture" event.

3.3 Dynamic Agent Desynchronization

Challenge: Creating custom agents with spaces in their names (e.g., "Software Engineer") caused the UI to silently fail during streaming because the backend formatted IDs differently than the frontend.

Solution: Discovered during penetration testing and auditing; unified the ID mapping logic across the stack using global string replacement (`replaceAll(' ', '_')`) and robust Pydantic constraints.

3.4 Free Tier API Rate Limiting & Failures

Challenge: Hitting OpenRouter's free tier endpoints simultaneously for 4 agents resulted in 429 Too Many Requests errors. Furthermore, the primary search engine (DuckDuckGo) frequently blocked automated requests.

Solution: Staggered the initial agent analysis using `asyncio.sleep` delays. Replaced the asynchronous DuckDuckGo integration with a highly stable thread-safe synchronous wrapper, and built a seamless fallback to the Tavily API if DuckDuckGo fails.

4. Future Roadmap

- **Persistent Database Analytics:** Expand the SQLite implementation to store and query historical debate metrics, allowing users to review past verdicts and export them to PDF dynamically.
- **PostgreSQL Migration:** Migrate from SQLite to a dedicated PostgreSQL instance for distributed deployment and massive scalability.
- **Mobile Responsiveness:** Enhance the Tailwind CSS styling to ensure the UI is fully functional and aesthetic on iOS and Android devices.